

Caccia al tesoro

Limite di tempo: 8 s Limite di memoria: 256 MiB

Stai cercando un tesoro perduto da tempo su un enorme griglia di $N \times N$ quadrati. Ci sono in totale $K \leq 3$ scrigni del tesoro nascosti in varie celle distinte del campo. Il tuo obiettivo è semplice: trovarne il più possibile con l'aiuto di una bussola magica.

Quando posizioni la bussola su una delle celle, essa troverà magicamente tutti i cammini di lunghezza minima verso uno scrigno del tesoro a distanza minima che utilizzano solo spostamenti a sinistra, a destra, in alto e in basso. Mostrerà quindi le direzioni di **tutti** gli spostamenti che siano il primo spostamento di un percorso verso uno scrigno di lunghezza minima. Se la cella contiene un tesoro, la bussola indicherà invece "TREASURE".

Quando trovi uno scrigno, ne saccheggia il tesoro ma non rimuovi lo scrigno perché è troppo pesante. Per questo la bussola continuerà a considerare anche i tesori già trovati quando trova i percorsi minimi.

Trova la posizione di tutti gli scrigni, utilizzando la bussola quanto meno possibile.

Task

Questo è un task interattivo. In ciascun testcase (cioè ciascuna esecuzione), il tuo programma dovrà risolvere diverse cacce al tesoro. Dovrai interagire con il grader utilizzando una libreria fornita dagli organizzatori, che contiene le seguenti dichiarazioni:

- **void** *NextHunt*(**int** &*N*, **int** &*K*): chiama questa funzione per iniziare una caccia al tesoro (all'inizio dell'esecuzione o dopo aver terminato quella precedente). La funzione scriverà la dimensione della griglia in *N* e il numero di scrigni in *K*. Se non ci sono più cacce al tesoro da risolvere nell'esecuzione corrente, la funzione imposterà invece $N = K = -1$. In tal caso, termina l'esecuzione del tuo programma. **Nota bene:** Puoi chiamare questa funzione prima di aver trovato tutti i tesori della caccia corrente, se il tuo obiettivo è solo ottenere un punteggio parziale.
- **enum** { *TREASURE* = 0, *DIR_RIGHT* = 1, *DIR_UP* = 2, *DIR_LEFT* = 4, *DIR_DOWN* = 8 }; costanti utilizzate in quanto segue
- **int** *Query*(**int** *x*, **int** *y*): Se la cella alle coordinate (*x*, *y*) contiene un tesoro, questa funzione restituisce *TREASURE*. Altrimenti, restituisce la somma di una o più costanti tra *DIR_RIGHT*, *DIR_UP*, *DIR_LEFT* e *DIR_DOWN*, corrispondenti alle mosse restituite dalla bussola quando viene posizionata sulla cella (*x*, *y*). Le coordinate *x* e *y* devono essere interi compresi tra 0 e $N - 1$. (Nota: in questo task, le coordinate *y* aumentano dall'alto verso il basso.)

Dopo che *NextHunt* ha impostato $N = K = -1$, il tuo programma non deve più chiamare né *NextHunt* né *Query*. Non deve inoltre chiamare *Query* prima della prima chiamata a *NextHunt*.

Se il tuo programma:

- non rispetta questi vincoli, o

- effettua più di 1000 query all'interno di una singola caccia al tesoro, o
- fornisce valori fuori dai limiti per x e/o y quando chiama `Query`

la libreria terminerà il tuo programma e otterrai il verdetto runtime-error (RTE) per il caso di prova corrente.

Un tesoro si considera trovato se hai effettuato una query sulla cella che lo contiene almeno una volta.

Per accedere alla libreria, il tuo programma deve includere l'header file `treasurehuntlib.h`:

```
#include "treasurehuntlib.h"
```

Puoi scaricare questo header file qui: `treasurehuntlib.h`.

Per aiutarti nello sviluppo della tua soluzione, è disponibile una semplice implementazione della libreria qui: `treasurehuntlib-public.cpp`. Per compilarla insieme al tuo programma, aggiungi semplicemente il suo nome come parametro al compilatore, ad esempio:

```
g++ sol.cpp treasurehuntlib-public.cpp
```

se `sol.cpp` è il nome del file contenente la tua soluzione.

L'implementazione in `treasurehuntlib-public.cpp` supporta, in aggiunta alle dichiarazioni sopra menzionate, un'ulteriore funzione:

- `void InitFromFile(const char *fileName)`: questa funzione legge un elenco di cacce al tesoro da un file ed esegue la tua soluzione con esse invece di generarle casualmente. Troverai maggiori dettagli all'interno dello stesso `treasurehuntlib-public.cpp`.

L'effettiva valutazione verrà effettuata con un'implementazione diversa da quella d'esempio. Puoi comunque assumere che non inquina il namespace globale con dichiarazioni diverse da quelle sopra elencate.

Il tuo codice **non può** leggere dallo standard input né scrivere sullo standard output.

Input

Assunzioni

- $2 \leq N \leq 10^6$
- $1 \leq K \leq 3$
- All'interno di una singola esecuzione del tuo programma, ci saranno al massimo 100 000 cacce al tesoro.
- All'interno di una singola caccia al tesoro, puoi effettuare al massimo 1000 query.
- Il sistema di valutazione non è adattivo.

Subtask

- Subtask 1 (10 punti): $K = 1$
- Subtask 2 (30 punti): $K = 2$
- Subtask 3 (60 punti): $K = 3$.

Punteggio

Un subtask può consistere di diversi casi di prova (diverse esecuzioni del tuo programma), e ciascun caso di prova può consistere di diverse cacce al tesoro. Ai fini del punteggio, tutte le cacce al tesoro di un determinato subtask vengono valutate insieme, indipendentemente da come erano state originariamente suddivise tra i casi di prova. Per la i -esima caccia al tesoro, indichiamo la dimensione della griglia con $N_i \times N_i$, il numero di tesori con K_i , il numero di query effettuate dal tuo programma con Q_i e il numero di tesori trovati dal tuo programma con F_i . Indichiamo inoltre con S il numero totale di punti disponibili per questo subtask. Allora il numero di punti assegnati al tuo programma per questo subtask è:

- Se il tuo programma ha sempre trovato tutti i tesori (cioè se $F_i = K_i$ per ogni i), il suo punteggio dipende da $t_i = \frac{Q_i}{\lceil \log_2 N_i \rceil}$:

$$\frac{S}{2} + \frac{S}{2} \cdot \min_i f(t_i) \text{ punti, dove } f(t_i) = \begin{cases} 1, & t_i \leq 11 \\ 1 - (t_i - 11)/9, & 11 \leq t_i \leq 20 \\ 0, & t \geq 20. \end{cases}$$

- Se il tuo programma non ha sempre trovato tutti i tesori (cioè se esiste un i tale per cui $F_i < K_i$), riceve:

$$\frac{S}{2} \cdot \min_i \frac{F_i}{K_i} \text{ punti.}$$

In altre parole, si ottiene metà dei punti per aver trovato tutti i tesori, e l'altra metà per averli trovati nel minor numero possibile di query. Per un punteggio pieno, la tua soluzione dovrebbe trovare tutti i tesori in al massimo $11 \lceil \log_2 N_i \rceil$ query. Tra $11 \lceil \log_2 N_i \rceil$ e $20 \lceil \log_2 N_i \rceil$ query, il punteggio decresce linearmente; e se si effettuano più di $20 \lceil \log_2 N_i \rceil$ query, la tua soluzione otterrà solo la prima metà dei punti per aver trovato tutti i tesori.

Se le formule sopra indicate portano a un numero non intero di punti per il subtask, questo verrà arrotondato all'intero più vicino.

Esempio

Chiamata	Valore di ritorno
<code>NextHunt(N, K)</code>	$N = 4, K = 1$
<code>Query(2, 0)</code>	<code>DIR_DOWN</code> + <code>DIR_RIGHT</code> = 9
<code>Query(3, 1)</code>	<code>DIR_DOWN</code>
<code>Query(3, 2)</code>	<code>TREASURE</code>

<code>NextHunt(N, K)</code>	$N = -1, K = -1$
-----------------------------	------------------
